

IMPLEMENTASI JARINGAN PERMISSIONED BLOCKCHAIN MEDBLOCK MENGGUNAKAN HYPERLEDGER FABRIC

Diajukan untuk memenuhi nilai matakuliah Pemrograman Berbasis Platform

Dosen Pengampu

Archy Renaldy Pratama Nugraha, S. Kom., M.T.



Disusun Oleh:

Habibi Muhammad Sejati	(105240040)
Mochamad Nurziad	(105240022)
Mohammad Reiki Azhar Zamzamy	(105240009)

**PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
INTERNATIONAL WOMEN UNIVERSITY**

2026

BAB I

DESAIN ARSITEKTUR KONSORSIUM & TOPOLOGI JARINGAN

1.1. Penjelasan Studi Kasus

Sistem ini dikembangkan sebagai implementasi *Permissioned Enterprise Blockchain Traceability* menggunakan platform *Hyperledger Fabric*. Studi kasus yang diangkat pada proyek ini adalah sistem pelacakan dan pengiriman rekam medis antar-rumah sakit (MedBlock). Di dunia nyata, pertukaran data medis pasien antar-rumah sakit seringkali terhambat oleh masalah keamanan data, dan kurangnya rasa saling percaya antar-instansi. Dengan memanfaatkan arsitektur blockchain konsorsium, proyek ini membuat pencatatan rekam medis yang tidak dapat diubah, transparan bagi pihak yang berwenang, namun tetap mengedepankan privasi data pasien antar rumah sakit. Fitur utama dalam sistem ini meliputi registrasi rekam medis baru, rujukan pasien antar rumah sakit, pelacakan jejak riwayat audit data dari ledger, serta modul tagihan rumah sakit yang bersifat rahasia.

1.2. Batasan Wewenang Masing-masing Organization

Sistem MedBlock mengonfigurasi 2 (dua) *Organization* rumah sakit yang tergabung dalam satu jaringan konsorsium, yaitu Rumah Sakit Welas Asih dan Rumah Sakit Muhammadiyah. Untuk menjaga kerahasiaan dan integritas data, batasan wewenang ditetapkan sebagai berikut:

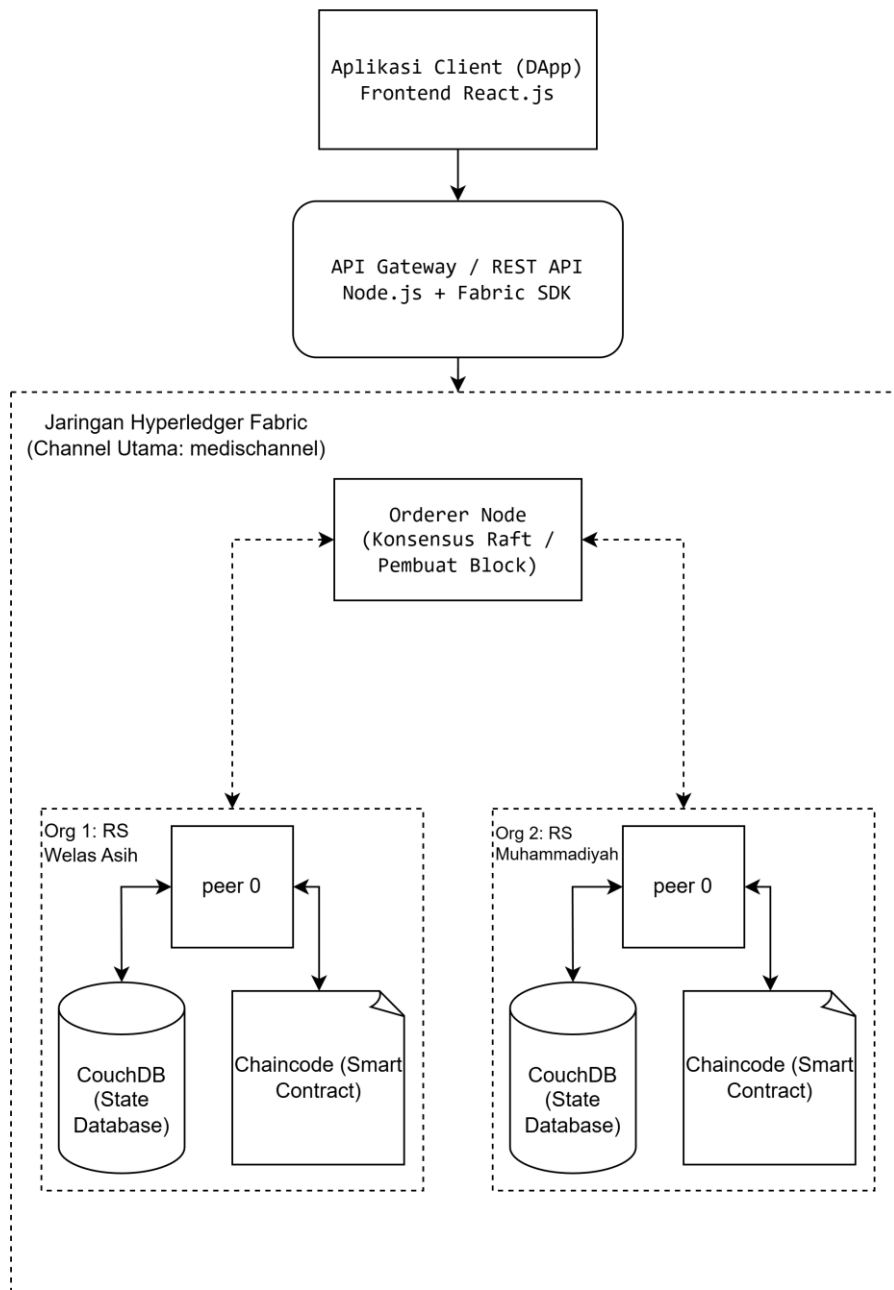
- Wewenang Data Publik: Kedua organisasi memiliki hak akses untuk melakukan *CreateAsset* (mendaftarkan pasien baru) dan *GetAssetHistory* (membaca riwayat medis dan diagnosa pasien) di saluran utama. Kedua organisasi juga memiliki wewenang untuk melakukan fungsi *TransferAsset* (merujuk pasien) hanya jika organisasi tersebut adalah pemilik sah (*Asset Owner*) dari rekam medis pasien pada saat itu.
- Wewenang Data Privat (*Private Data Collection / PDC*): Sistem menerapkan kebijakan PDC untuk mengamankan data finansial yang sensitif.
 - RS Welas Asih: Hanya berwenang membuat, mengenkripsi, dan membaca rincian tagihan medis milik pasien yang dirawat di RS Welas Asih. Sistem secara otomatis akan memblokir RS Welas Asih jika mencoba mengakses brankas data finansial milik RS Muhammadiyah.
 - RS Muhammadiyah: Hanya berwenang membuat, mengenkripsi, dan membaca rincian tagihan medis milik pasien yang dirawat di RS Muhammadiyah, serta tidak memiliki hak akses untuk melihat tagihan finansial milik RS Welas Asih.

1.3. Pembagian Tugas Antar Anggota Kelompok

Proyek ini dikembangkan oleh tiga anggota kelompok dengan pembagian peran sebagai berikut:

Nama Anggota	Peran dalam Tim	Tanggung Jawab Utama
Habibi	Infrastruktur, DevOps & Chaincode Developer	Bertanggung jawab menyusun berkas konfigurasi jaringan (docker-compose.yaml, configtx.yaml, pembuatan kriptografi MSP/Sertifikat menggunakan cryptogen). Serta Bertanggung jawab menulis kode pemrograman logika bisnis pelacakan (Chaincode) menggunakan bahasa Go (Golang).
Nurziad	Backend Developer	Bertanggung jawab membangun REST API server menggunakan Node.js/Go SDK untuk menjembatani komunikasi antara aplikasi luar dengan jaringan Hyperledger Fabric.
Reiki	Frontend & QA Tester	Bertanggung jawab membuat antarmuka aplikasi (DApp web panel) untuk simulasi interaksi aktor dan melakukan pengujian fungsionalitas sistem (End-to-End).

1.4. Topologi Jaringan



BAB II

TUTORIAL INSTALASI JARINGAN & CODE PROGRAM

2.1. Langkah Instalasi Prasyarat Lingkungan

Dalam membangun jaringan *blockchain* MedBlock, terdapat beberapa perangkat lunak prasyarat utama yang harus disiapkan terlebih dahulu pada lingkungan pengembangan, antara lain:

- WSL2 & Ubuntu: Disiapkan sebagai subsistem Linux virtual untuk mengeksekusi perintah-perintah jaringan dasar.
- Docker & Docker Compose: Disiapkan sebagai penampung mesin (container engine) utama. Docker bertugas menjalankan node jaringan (Peer, Orderer, dan CouchDB) secara terisolasi.
- Golang & Node.js: Lingkungan pemrograman Golang dipersiapkan untuk menulis logika Smart Contract (Chaincode), sedangkan Node.js digunakan untuk membangun backend REST API.
- Hyperledger Fabric Binaries: Kumpulan utilitas bawaan (seperti cryptogen dan configtxgen) yang disiapkan untuk mencetak sertifikat keamanan dan konfigurasi blok jaringan.

2.2. Konfigurasi & Inisialisasi Jaringan Manual

Setelah prasyarat terpenuhi, langkah selanjutnya adalah membangun fondasi infrastruktur jaringan secara manual:

1. Pembuatan Kriptografi MSP & Sertifikat:
 - Siapkan file `crypto-config.yaml` yang mendefinisikan struktur organisasi (RS Welas Asih dan RS Muhammadiyah) beserta jumlah Peer dan Orderer.
 - Eksekusi perintah `cryptogen generate --config=./crypto-config.yaml` untuk mencetak seluruh dompet digital, Private Key, dan sertifikat untuk masing-masing node secara otomatis.
2. Pembuatan Aturan Jaringan (Channel & Konsensus):
 - Siapkan file `configtx.yaml` yang berisi aturan pembentukan blok (konsensus Raft), batasan organisasi, dan kebijakan hak akses (ACL).
 - Buat sebuah direktori khusus (misal: `channel-artifacts`) untuk menyimpan hasil cetakan konfigurasi jaringan.
3. Pencetakan Genesis Block dan Channel Transaction:
 - Gunakan utilitas Fabric dengan mengeksekusi `configtxgen -profile MedBlockOrdererGenesis -channelID system-channel -outputBlock ./channel-artifacts/genesis.block` untuk membuat blok pertama (Genesis Block) sebagai inti dari Orderer.

- Cetak fail transaksi pembuatan saluran utama jaringan dengan perintah
`configtxgen -profile MedBlockChannel -outputCreateChannelTx
./channel-artifacts/medischannel.tx -channelID medischannel.`
4. Menjalankan Jaringan via Docker:
- Setelah seluruh sertifikat dan blok konfigurasi siap, jalankan seluruh penampung (container) jaringan (Orderer, Peer, dan CouchDB) secara bersamaan menggunakan konfigurasi `docker-compose.yaml` dengan perintah `docker-compose up -d`.

2.3 Panduan Pembuatan Kanal dan Chaincode Lifecycle

Setelah infrastruktur *container* Docker berjalan, jaringan belum dapat memproses transaksi sebelum kanal komunikasi dibentuk dan *Smart Contract* (Chaincode) ditanamkan melalui prosedur *Fabric Lifecycle*. Berikut adalah urutan eksekusinya:

1. Inisialisasi dan Bergabung ke Kanal (Channel Join)

```
# Set jadi Welas Asih
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="RSWelasAsihMSP"
export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/crypto-
config/peerOrganizations/welasasih.mednet.com/peers/peer0.welasasih.medn
et.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=${PWD}/crypto-
config/peerOrganizations/welasasih.mednet.com/users/Admin@welasasih.me
dnet.com/msp
export CORE_PEER_ADDRESS=localhost:7051
export
CORE_PEER_TLS_SERVERHOSTOVERRIDE=peer0.welasasih.mednet.co
m
```

```
# Welas Asih Bikin Channel (dengan override hostname)
```

```
peer channel create -o localhost:7050 -c medischannel -f ./channel-
artifacts/medischannel.tx --outputBlock ./channel-
artifacts/medischannel.block --tls --cafile ${PWD}/crypto-
config/ordererOrganizations/mednet.com/orderers/orderer.mednet.com/msp/tl
scacerts/tlsca.mednet.com-cert.pem --ordererTLSHostnameOverride
orderer.mednet.com
```

```
# Welas Asih Join
```

```
peer channel join -b ./channel-artifacts/medischannel.block
```

```
# Set jadi Muhammadiyah & Join
```

```
export CORE_PEER_LOCALMSPID="RSMuhammadiyahMSP"
```

```
export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/crypto-
config/peerOrganizations/muhammadiyah.mednet.com/peers/peer0.muhamm
adiyah.mednet.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=${PWD}/crypto-
config/peerOrganizations/muhammadiyah.mednet.com/users/Admin@muha
mmadiyah.mednet.com/msp
export CORE_PEER_ADDRESS=localhost:9051
export
CORE_PEER_TLS_SERVERHOSTOVERRIDE=peer0.muhammadiyah.med
net.com
```

```
# Muhammadiyah Join
peer channel join -b ./channel-artifacts/medischannel.block
```

2. Pengemasan dan Instalasi Chaincode (Package & Install)

```
# Bungkus Kode Go (Package)
peer lifecycle chaincode package rekam-medis.tar.gz --path ./chaincode-
rekam-medis --lang golang --label rekam-medis_1.0
```

```
# Install di Muhammadiyah (Karena terminal masih pakai profil
Muhammadiyah)
peer lifecycle chaincode install rekam-medis.tar.gz
```

```
# Ganti jadi Welas Asih & Install
export CORE_PEER_LOCALMSPID="RSWelasAsihMSP"
export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/crypto-
config/peerOrganizations/welasasih.mednet.com/peers/peer0.welasasih.medn
et.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=${PWD}/crypto-
config/peerOrganizations/welasasih.mednet.com/users/Admin@welasasih.me
dnet.com/msp
export CORE_PEER_ADDRESS=localhost:7051
export
CORE_PEER_TLS_SERVERHOSTOVERRIDE=peer0.welasasih.mednet.co
m
```

```
peer lifecycle chaincode install rekam-medis.tar.gz
```

3. Mencari Package ID

```
#untuk mengecek package ID
peer lifecycle chaincode queryinstalled
```

```
#untuk mempermudah approve chaincode
export CC_PACKAGE_ID=PASTE_DI_SINI
```

4. Approve Chaincode

```
# Approve dari Welas Asih
peer lifecycle chaincode approveformyorg -o localhost:7050 --
ordererTLSHostnameOverride orderer.mednet.com --channelID
medischannel --name rekam-medis --version 1.0 --package-id
$CC_PACKAGE_ID --sequence 1 --tls --cafile ${PWD}/crypto-
config/ordererOrganizations/mednet.com/orderers/orderer.mednet.com/msp/tl
scacerts/tlsca.mednet.com-cert.pem --collections-config ./chaincode-rekam-
medis/collections_config.json
```

```
# Ganti profil ke Muhammadiyah & Approve
export CORE_PEER_LOCALMSPID="RSMuhammadiyahMSP"
export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/crypto-
config/peerOrganizations/muhammadiyah.mednet.com/peers/peer0.muhamm
adiyah.mednet.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=${PWD}/crypto-
config/peerOrganizations/muhammadiyah.mednet.com/users/Admin@muha
mmadiyah.mednet.com/msp
export CORE_PEER_ADDRESS=localhost:9051
export
CORE_PEER_TLS_SERVERHOSTOVERRIDE=peer0.muhammadiyah.med
net.com
```

```
peer lifecycle chaincode approveformyorg -o localhost:7050 --
ordererTLSHostnameOverride orderer.mednet.com --channelID
medischannel --name rekam-medis --version 1.0 --package-id
$CC_PACKAGE_ID --sequence 1 --tls --cafile ${PWD}/crypto-
config/ordererOrganizations/mednet.com/orderers/orderer.mednet.com/msp/tl
scacerts/tlsca.mednet.com-cert.pem --collections-config ./chaincode-rekam-
medis/collections_config.json
```

5. Commit Chaincode

```
# Daftarkan Domain ke Linux (menggunakan password linux)
echo "127.0.0.1 peer0.welasasih.mednet.com
peer0.muhammadiyah.mednet.com orderer.mednet.com" | sudo tee -a
/etc/hosts
```

```
# Membersihkan profil
unset CORE_PEER_TLS_SERVERHOSTOVERRIDE
```

```
# Commit Menggunakan Domain Asli
peer lifecycle chaincode commit -o orderer.mednet.com:7050 --channelID
medischannel --name rekam-medis --version 1.0 --sequence 1 --tls --cafile
${PWD}/crypto-
config/ordererOrganizations/mednet.com/orderers/orderer.mednet.com/msp/tl
```



```

scacerts/tlsca.mednet.com-cert.pem --peerAddresses
peer0.welasasih.mednet.com:7051 --tlsRootCertFiles ${PWD}/crypto-
config/peerOrganizations/welasasih.mednet.com/peers/peer0.welasasih.medn
et.com/tls/ca.crt --peerAddresses peer0.muhammadiyah.mednet.com:9051 --
tlsRootCertFiles ${PWD}/crypto-
config/peerOrganizations/muhammadiyah.mednet.com/peers/peer0.muhamm
adiyah.mednet.com/tls/ca.crt --collections-config ./chaincode-rekam-
medis/collections_config.json

```

6. Anchor peer

```
# download core.yaml
```

```
wget https://raw.githubusercontent.com/hyperledger/fabric/release-2.2/sampleconfig/core.yaml
```

```
export FABRIC_CFG_PATH=${PWD}
```

```
# 1. Cetak File Anchor
```

```
configtxgen -profile MedisChannel -outputAnchorPeersUpdate ./channel-
artifacts/WelasAsihMSPanchors.tx -channelID medischannel -asOrg
RSWelasAsihMSP
```

```
configtxgen -profile MedisChannel -outputAnchorPeersUpdate ./channel-
artifacts/MuhammadiyahMSPanchors.tx -channelID medischannel -asOrg
RSMuhammadiyahMSP
```

```
# 2. Update Anchor Welas Asih
```

```
export CORE_PEER_TLS_ENABLED=true
```

```
export CORE_PEER_LOCALMSPID="RSWelasAsihMSP"
```

```
export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/crypto-
config/peerOrganizations/welasasih.mednet.com/peers/peer0.welasasih.medn
et.com/tls/ca.crt
```

```
export CORE_PEER_MSPCONFIGPATH=${PWD}/crypto-
config/peerOrganizations/welasasih.mednet.com/users/Admin@welasasih.me
dnet.com/msp
```

```
export CORE_PEER_ADDRESS=localhost:7051
```

```
export
```

```
CORE_PEER_TLS_SERVERHOSTOVERRIDE=peer0.welasasih.mednet.co
m
```

```
peer channel update -o localhost:7050 --ordererTLSHostnameOverride
orderer.mednet.com -c medischannel -f ./channel-
```

```
artifacts/WelasAsihMSPanchors.tx --tls --cafile ${PWD}/crypto-
config/ordererOrganizations/mednet.com/orderers/orderer.mednet.com/msp/tl
scacerts/tlsca.mednet.com-cert.pem
```

```
# 3. Update Anchor Muhammadiyah
```

```
export CORE_PEER_LOCALMSPID="RSMuhammadiyahMSP"
```

```
export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/crypto-
config/peerOrganizations/muhammadiyah.mednet.com/peers/peer0.muhamm
adiyah.mednet.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=${PWD}/crypto-
config/peerOrganizations/muhammadiyah.mednet.com/users/Admin@muha
mmadiyah.mednet.com/msp
export CORE_PEER_ADDRESS=localhost:9051
export
CORE_PEER_TLS_SERVERHOSTOVERRIDE=peer0.muhammadiyah.med
net.com
peer channel update -o localhost:7050 --ordererTLSHostnameOverride
orderer.mednet.com -c medischannel -f ./channel-
artifacts/MuhammadiyahMSPanchors.tx --tls --cafile ${PWD}/crypto-
config/ordererOrganizations/mednet.com/orderers/orderer.mednet.com/msp/tl
scacerts/tlsca.mednet.com-cert.pem
```

2.4. Penjelasan Potongan Kode Chaincode & Integrasi Fabric SDK

Sistem *MedBlock* menggunakan *Smart Contract* (Chaincode) yang ditulis menggunakan bahasa pemrograman Golang (contractapi). Chaincode ini bertanggung jawab atas seluruh logika bisnis pelacakan, manajemen privasi, dan validasi data sebelum dicatat ke dalam buku besar (*ledger*).

1. Logika Chaincode Utama (Golang)

1.1 Struktur Data Aset (Structs)

Sistem mendefinisikan bentuk data menggunakan *Struct*. Terdapat pemisahan tegas antara data rekam medis yang bersifat publik (dapat dilihat seluruh RS di jaringan) dan data tagihan yang bersifat privat (PDC).

// Struct Aset Publik (Bisa dilihat kedua RS)

```
type RekamMedis struct {
    IDPasien    string `json:"idPasien"`
    NamaPasien  string `json:"namaPasien"`
    RiwayatSakit string `json:"riwayatSakit"`
    Tindakan    string `json:"tindakan"`
    Pemilik     string `json:"pemilik" // Menandakan RS yang merawat
    saat ini
}
```

// Struct Aset Privat / PDC (Hanya dilihat RS tertentu)

```
type TagihanPrivate struct {
    IDPasien    string `json:"idPasien"`
    TotalTagihan int    `json:"totalTagihan"`
    DetailObat  string `json:"detailObat"`
}
```

1.2 Fungsi Registrasi Aset Publik (Create & Read)

Fungsi pendaftaran pasien baru dan pembacaan data tunggal dilakukan menggunakan API standar Fabric, yaitu PutState untuk menyimpan dan GetState untuk membaca dari *World State*.

// Menyimpan data pasien baru ke ledger

```
func (s *SmartContract) CreateRekamMedis(...) error {  
    rm := RekamMedis{ ... }  
    rmJSON, err := json.Marshal(rm)  
    // PutState menyimpan pasangan Key-Value ke State Database  
    return ctx.GetStub().PutState(idPasien, rmJSON)  
}
```

1.3 Fungsi Pencarian Parametrik (Rich Query CouchDB)

Untuk menampilkan seluruh daftar pasien di tabel *Dashboard* tanpa harus mencari ID-nya satu per satu, sistem memanfaatkan fitur *Rich Query* khusus CouchDB menggunakan GetQueryResult.

```
func (s *SmartContract) GetAllRekamMedis(ctx  
contractapi.TransactionContextInterface) ([]*RekamMedis, error) {  
    // Sintaks JSON Selector CouchDB: Mencari semua data yang ID-  
    nya tidak kosong  
    queryString := `{"selector":{"idPasien":{"$ne":""}}}`  
  
    resultsIterator, err := ctx.GetStub().GetQueryResult(queryString)  
    // ... iterasi resultsIterator.HasNext() dan konversi ke list JSON  
}
```

1.4 Fungsi Rujuk Pasien (Transfer Asset)

Fungsi ini mensimulasikan rujukan pasien antar-instansi. Logikanya adalah memanggil data lama menggunakan ReadRekamMedis, mengubah *value* pada atribut Pemilik (misal dari RS Welas Asih menjadi RS Muhammadiyah), lalu menyimpannya kembali menimpa *state* yang lama.

```
func (s *SmartContract) RujukPasien(ctx  
contractapi.TransactionContextInterface, idPasien string, rsTujuan  
string) error {  
    rm, err := s.ReadRekamMedis(ctx, idPasien)  
    // Pindah penguasaan / lokasi (Transfer Asset)  
    rm.Pemilik = rsTujuan  
    rmJSON, _ := json.Marshal(rm)  
    return ctx.GetStub().PutState(idPasien, rmJSON)  
}
```

1.5 Fungsi Pelacakan Riwayat (Audit Trail)

Setiap kali data rekam medis diperbarui (misalnya saat dirujuk), Fabric otomatis menyimpan riwayatnya secara *immutable*. Fungsi ini menggunakan `GetHistoryForKey` untuk menarik rekam jejak tersebut.

```
func (s *SmartContract) GetRiwayatPasien(ctx
contractapi.TransactionContextInterface, idPasien string)
([]*RiwayatRekamMedis, error) {
    // Menarik seluruh riwayat perubahan aset dari masa lalu hingga
    sekarang
    resultsIterator, err := ctx.GetStub().GetHistoryForKey(idPasien)
    // ...
    // Mengkonversi format Timestamp bawaan Fabric ke waktu
    kalender normal
    timestamp := time.Unix(response.Timestamp.Seconds,
int64(response.Timestamp.Nanos))
    // ...
}
```

1.6 Fungsi Keamanan Data Finansial (Private Data Collection / PDC)

Ini adalah lapisan keamanan tertinggi dalam sistem. Pembuatan data tagihan menggunakan `GetTransient()` agar nominal uang tidak terekam dalam *payload* transaksi publik. Data tersebut kemudian disimpan secara rahasia di brankas masing-masing RS menggunakan `PutPrivateData`.

```
// Create tagihan menggunakan Transient Data
func (s *SmartContract) CreateTagihanPrivate(ctx
contractapi.TransactionContextInterface, namaKoleksi string) error {
    transientMap, _ := ctx.GetStub().GetTransient()
    tagihanJSON := transientMap["tagihan_data"]

    // PutPrivateData: Menyimpan ke brankas rahasia sesuai organisasi
    (namaKoleksi)
    return ctx.GetStub().PutPrivateData(namaKoleksi, tagihan.IDPasien,
tagihanJSON)
}

// Read tagihan dari brankas rahasia
func (s *SmartContract) ReadTagihanPrivate(...) (*TagihanPrivate,
error) {
    // GetPrivateData: Menolak akses jika RS yang meminta tidak
    memiliki wewenang
    tagihanJSON, _ := ctx.GetStub().GetPrivateData(namaKoleksi,
idPasien)
    // ...
}
```

2. Implementasi Fabric SDK di Sisi Backend (Node.js)

Agar aplikasi klien (*Frontend React*) dapat berkomunikasi dengan *Chaincode* di dalam jaringan *blockchain*, sistem menggunakan *fabric-network* (Fabric SDK) berbasis Node.js yang bertindak sebagai *REST API Gateway*. Implementasi utamanya meliputi:

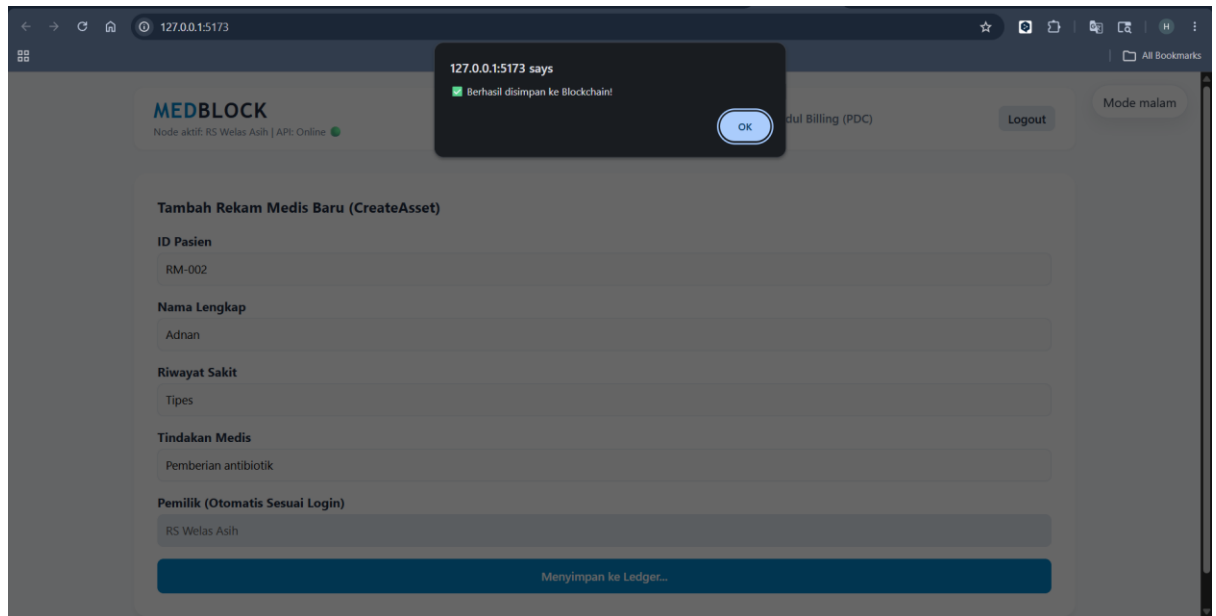
- Manajemen Dompet Kriptografi (Wallet): Backend mengelola identitas digital (X.509) secara dinamis menggunakan modul Wallets. Aplikasi akan memilih dompet (wallet Welas Asih atau wallet-muhammadiyah) berdasarkan identitas instansi pengguna yang sedang masuk (login).
- Gateway Connection: Backend membangun koneksi ke Peer menggunakan `gateway.connect()`. Melalui koneksi ini, aplikasi mendeteksi struktur jaringan menggunakan fitur Service Discovery (`discovery: { enabled: true, asLocalhost: true }`).
- Eksekusi Transaksi (Submit vs Evaluate): * Metode `submitTransaction` digunakan untuk fungsi mutasi state (seperti `CreateRekamMedis` dan `RujukPasien`), karena membutuhkan persetujuan (endorsement) dari Orderer.
 - Metode `evaluateTransaction` digunakan untuk fungsi baca murni (seperti `ReadRekamMedis` dan `GetAllRekamMedis`) sehingga respons dapat ditarik dengan instan tanpa harus membuang resource pembentukan blok baru.

BAB III

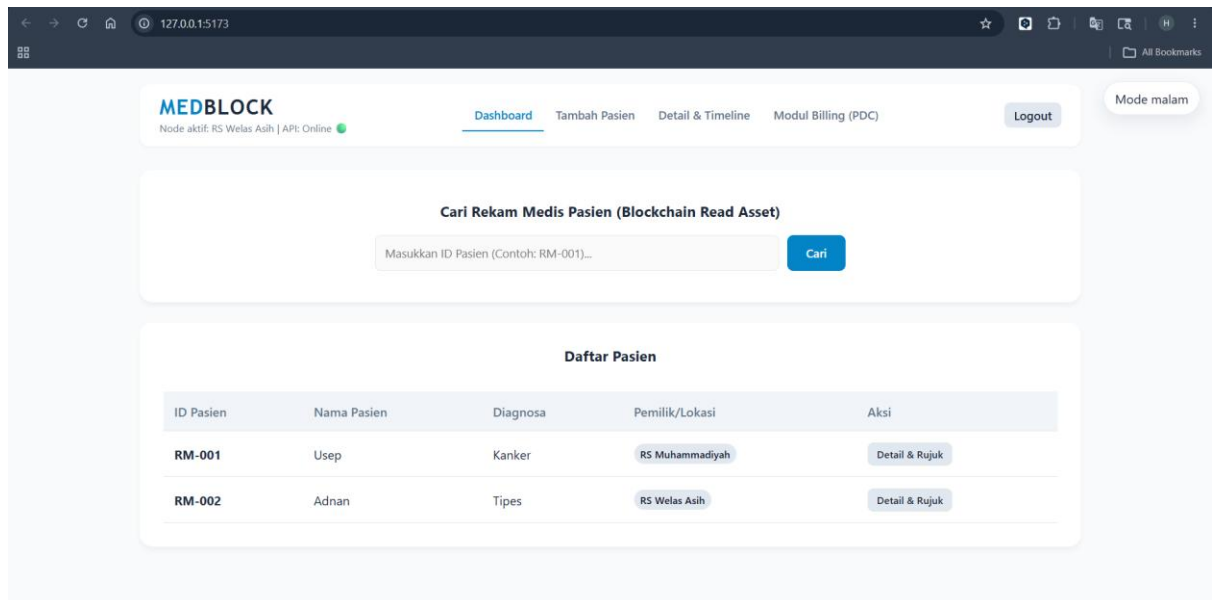
Pengujian Jaringan & Demo Kasus

3.1 Bukti Eksekusi Transaksi (Invoke & Query)

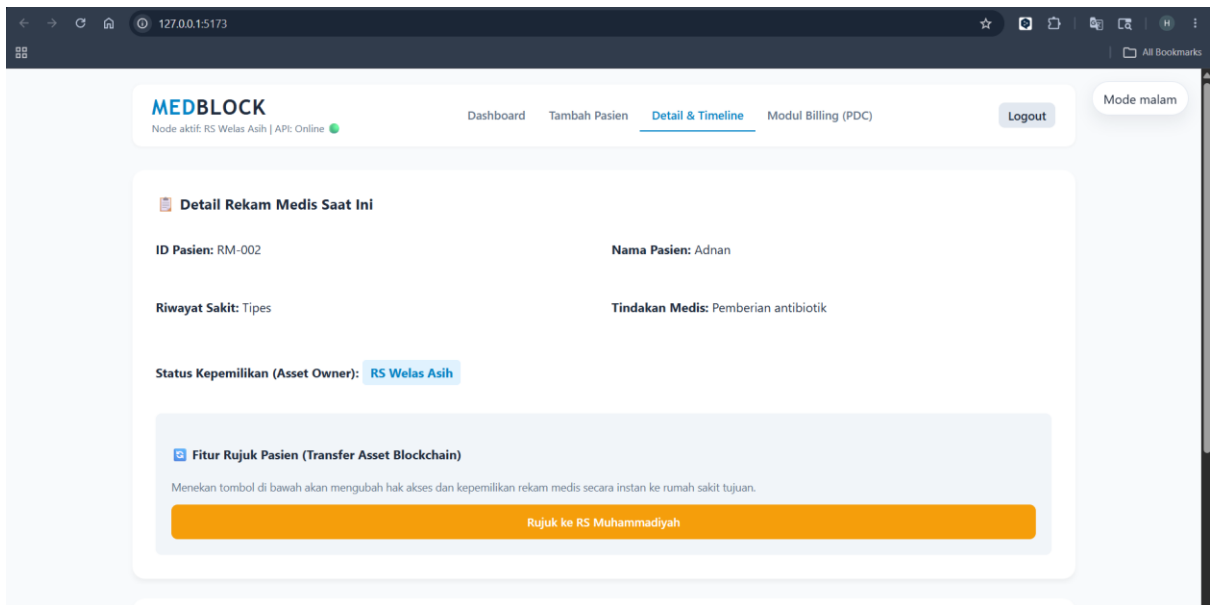
Fokus di sini adalah membuktikan bahwa fungsi *CreateAsset* (Tambah Data) dan *TransferAsset* (Rujuk Pasien) benar-benar bekerja.



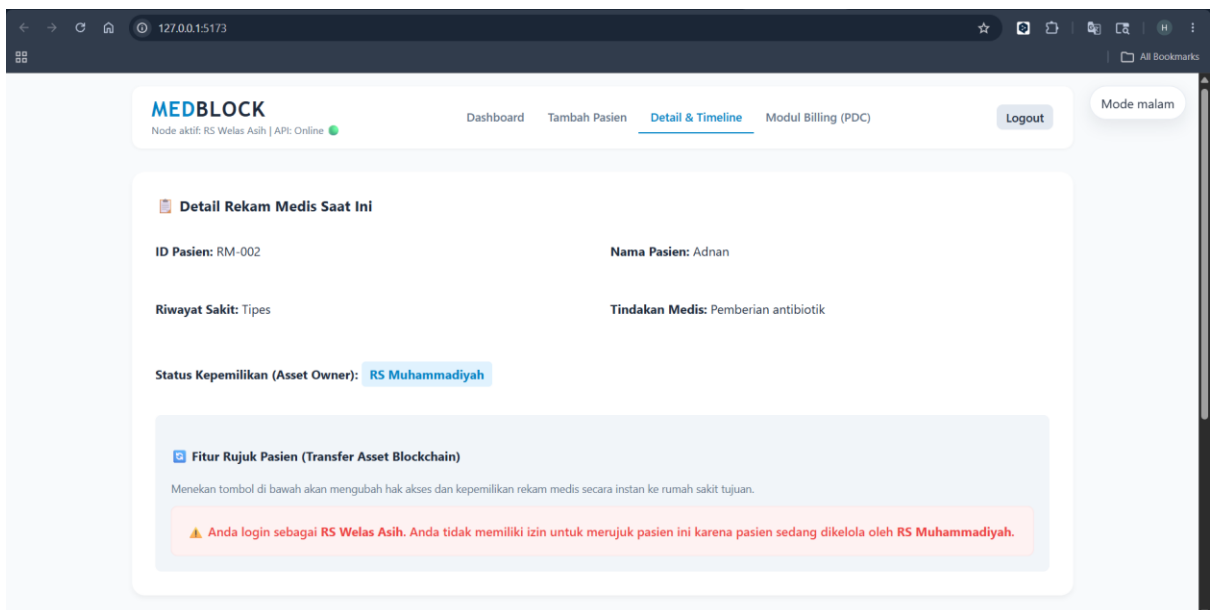
Hasil pemanggilan Query CouchDB menampilkan seluruh data aset medis pada ledger



Sebelum dirujuk ke Muhammadiyah



Setelah dirujuk ke muhammadiyah

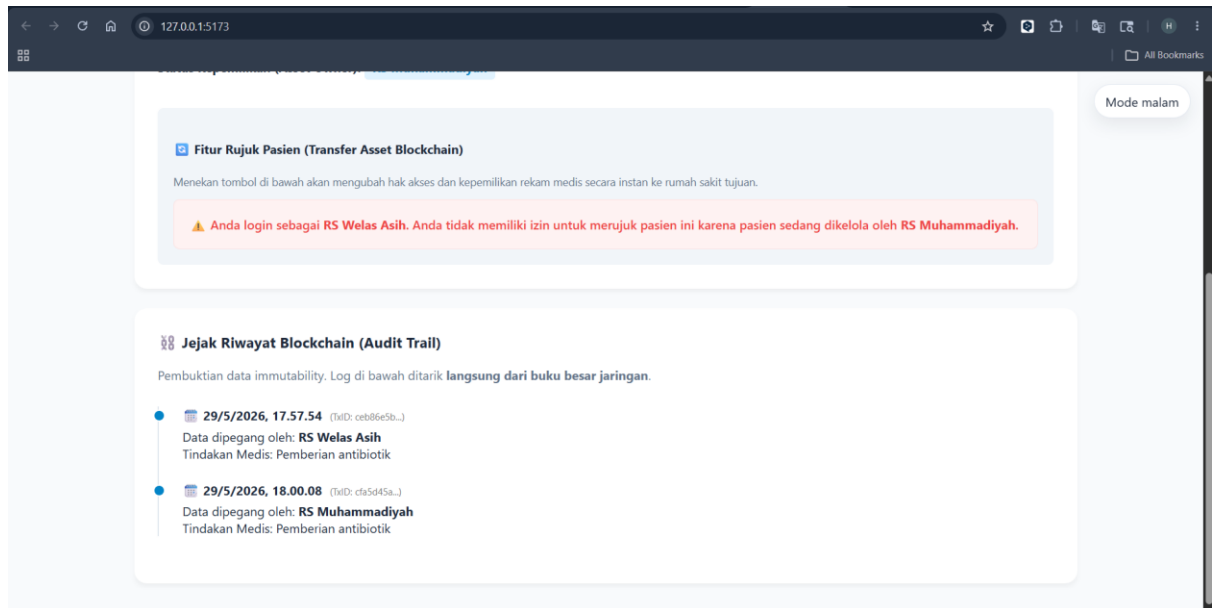


Log di terminal

```
• habi@kyouka:~/rekam-medis-rumahsakit$ cd backend
• habi@kyouka:~/rekam-medis-rumahsakit/backend$ node app.js
Menyiapkan koneksi ke RS Welas Asih...
✅ SUKSES: Backend berhasil terhubung ke Smart Contract Rekam Medis!
Menyiapkan koneksi ke RS Muhammadiyah...
✅ SUKSES: Backend Muhammadiyah terhubung ke Smart Contract!
🚀 REST API berjalan di http://localhost:3000
Menarik semua data pasien menggunakan Rich Query...
Menarik semua data pasien menggunakan Rich Query...
Memproses rujukan pasien RM-002 ke RS Muhammadiyah...
```

3.2 Log Audit Jejak Pelacakan

Timeline Riwayat Pasien



Data transaksi dari Smart Contract berhasil disimpan

